

PRISM — Integration with vLLM-SR

White Paper Contribution — Model Selection Section

Mossaab Souaissa

Reference: <https://doi.org/10.5281/zenodo.18750029>

1. Problem Statement

Modern LLM routing systems — including vLLM-SR — answer the question: which model is best suited to handle this request? The Signal-Decision Architecture selects the optimal model based on semantic signals and routing decisions. However, a critical gap remains unaddressed:

A selected model can still respond outside its domain **with no structural constraint preventing it from doing so**.

This produces confident hallucinations — a specialized model improvising answers outside its training domain. The problem is not routing accuracy; it is the absence of a legitimacy verification mechanism after selection.

System	Question answered	Limitation
vLLM-SR	Which model is best suited?	Does not verify whether the model is legitimate to respond
PRISM 153-key	Is this model legitimate to respond?	Does not perform routing optimization
vLLM-SR + PRISM	Both questions answered simultaneously	Complementary — non-redundant

2. The 153-key Protocol

The 153-key is not a single prompt — it is a sequential protocol composed of three distinct keys, each with a precise role and a structure adapted to that role. The three keys execute at different moments in the system lifecycle.

Key	Name	When	Role
Key 1	QUALIFICATION	Model initialization (once)	The model declares its domain, capabilities and score
Key 2	CLASSIFICATION	Every request — before routing	Identify the intent and target domain of the query
Key 3	EXECUTION	Every request — after routing	Force the selected model to accept or refuse

This sequencing is the core of the ZN protocol as adapted by PRISM: each key is an independent structural barrier. The system cannot short-circuit the execution order.

2.1 Key 1 — QUALIFICATION

When: at model initialization, once only	 	Feeds: the 153-Registry
---	----------	--------------------------------

The Qualification key forces each model to self-assess before being registered in the 153-Registry. The model examines its own vocabulary, knowledge and limits, then produces a structured declaration.

Operating principle

- The model receives the closed list of available domains (DOMAINS_LIST)
- It must select exactly one domain — or declare 'general' or 'unknown'
- It lists 2 to 5 tasks it can handle with confirmed expertise
- It lists 1 to 3 domains it must systematically refuse
- It assigns itself an expertise score from 0.00 to 1.00

Output format

```
{"domain": "quality_control", "can_process": ["measure AQL defects", "analyze non-conformities"],  
  "must_refuse": ["textile production", "machine maintenance"], "score": 0.88}
```

Scoring scale

Score	Level	Meaning
0.00 - 0.30	Superficial	General knowledge without specialization
0.31 - 0.60	Basic	Specialized vocabulary but limited capabilities
0.61 - 0.90	Confirmed	Real expertise with protocols and methods
0.91 - 1.00	Total mastery	Standards, formulas, complex cases — maximum expertise

Role in the 153-Registry

The Qualification output feeds directly into the 153-Registry — the dynamic directory of qualified models. Each entry contains the declared domain, score, capabilities and status (active, suspended, banned). This registry is what Keys 2 and 3 consult at runtime.

2.2 Key 2 — CLASSIFICATION

When: every request, before routing | **Produces: domain + confidence + keywords**

The Classification key analyzes the intent of each incoming request to identify the required domain of expertise. It never answers the question — it classifies only.

Operating principle

Absolute rule: Key 2 must NEVER answer the question asked. Its sole mission is to identify the target domain with a confidence level.

- It receives the user query and the list of available domains
- It extracts technical keywords present in the query
- It computes a confidence score from 0.00 to 1.00
- It determines whether the query requires specialized expertise (requires_expertise)
- It produces a classification JSON — no other text

Output format

```
{"domain": "industrial_maintenance", "confidence": 0.85,
  "keywords": ["MTBF", "preventive maintenance", "machine failure"], "requires_expertise": true}
```

Confidence scale

Confidence	Interpretation	Recommended action
0.00 - 0.30	Uncertain domain, ambiguous keywords	Re-classification or escalation to human operator
0.40 - 0.60	Probable domain, some indicators	Routing with low confidence warning
0.70 - 0.90	Clear domain, specific terminology	Normal routing to qualified model
1.00	Obvious domain, confirmed technical vocabulary	Priority routing, maximum confidence score

Difference from vLLM-SR routing

Key 2 — CLASSIFICATION		vLLM-SR Routing
Question answered	Which domain is required?	Which model is best suited?
Input	User query + DOMAINS_LIST	Semantic signal + candidates
Output	domain + confidence + keywords	Selected model
Answers the question?	Never	No — routes only

2.3 Key 3 — EXECUTION

When: every request, after vLLM-SR routing | **Produces:** ACCEPTED or REFUSED + response

The Execution key is the final barrier of the 153-key protocol. It forces the model selected by vLLM-SR to confirm its legitimacy before responding. This is the mechanism that makes refusal structurally impossible to bypass.

Operating principle

Absolute rule: the model must NEVER improvise outside its domain, extrapolate on related topics, or redirect to another expert. It accepts or it refuses — nothing else.

- The model receives its declared domain of expertise (from Qualification)
- It evaluates whether the query contains at least 2 technical terms from its domain
- It verifies whether an incorrect answer could cause technical or financial harm
- It produces an ACCEPTED JSON with its response, or REFUSED with a reason
- No text outside the JSON format — no exceptions

Acceptance criteria

A query is ACCEPTED if AND ONLY IF all three conditions are met:

- It contains at least 2 technical terms from the model's declared domain
- It requires specialized expertise to answer correctly
- An incorrect answer could cause technical or financial harm

A query is REFUSED if any of these conditions is true:

- It relies on general common sense without specific technical content
- It concerns a different specialized domain than the model's
- It is ambiguous regarding the target domain

Output format

```
If ACCEPTED:
{"status":"ACCEPTED","reason":"reason max 20 words","response":"detailed response here"}
```

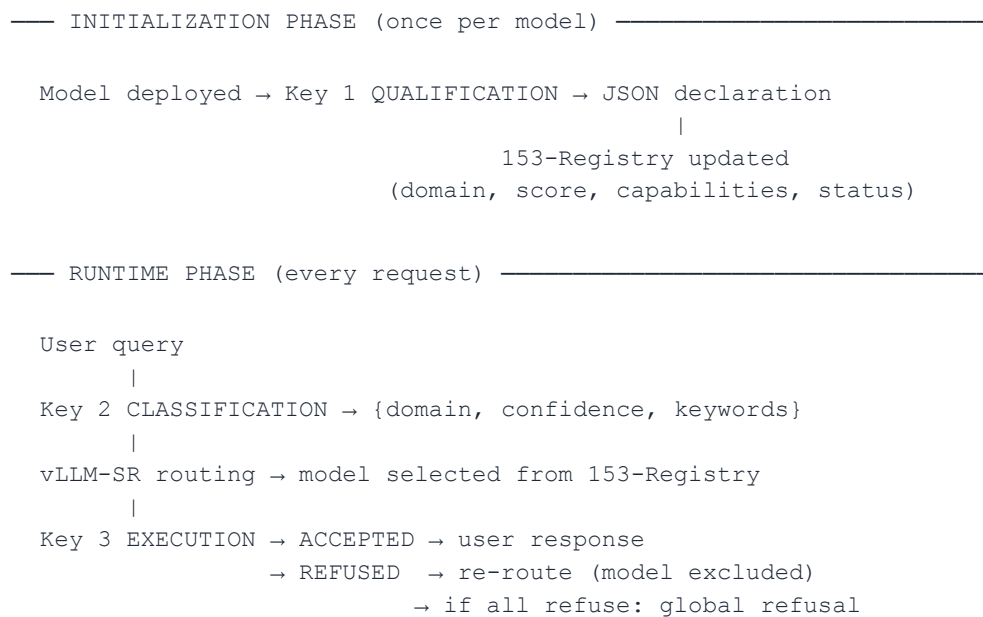
```
If REFUSED:
{"status":"REFUSED","reason":"reason max 20 words","response":""}
```

What a refusal triggers

A REFUSED from Key 3 is not an error — it is an architectural signal. It triggers the vLLM-SR re-routing mechanism with the refused model excluded from candidates, an empirical score update in the 153-Registry, and if all candidates refuse — a global refusal that is documented and traced.

2.4 The complete protocol — Overview

The three keys form a sequential protocol where each step is a necessary condition for the next. No key can be short-circuited.



Key	ZN Structure	Variable?	Output
Key 1 — QUALIFICATION	Mission + Context + Domains + Scoring + Format	No — fixed structure at init	JSON: domain + can_process + must_refuse + score
Key 2 — CLASSIFICATION	Role + Prohibitions + Domains + Confidence + Format	Yes — domains and glossary adaptable	JSON: domain + confidence + keywords + requires_expertise
Key 3 — EXECUTION	Domain + Rule + Prohibitions + Criteria + Format	Yes — domain injected dynamically	JSON: status + reason + response

Why the ZN structure is variable

The ZN protocol defines the encapsulation and strict prohibition principles that apply to all three keys. But the internal structure of each key is adapted to its specific role:

- Key 1 needs training context and a scoring scale — not a list of execution prohibitions

- Key 2 needs a domain glossary and confidence rules — not technical acceptance criteria
- Key 3 needs precise ACCEPTED/REFUSED criteria — not initialization context

ZN is an encapsulation framework, not a rigid template. Each key adapts it to its function while preserving the fundamental principle: *the model cannot bypass the protocol.*

3. The Three Integration Modes

PRISM integrates into vLLM-SR's selectModelFromCandidates pipeline in three configurable modes. Each mode activates a specific combination of the three 153-key protocol keys.

A fundamental principle applies to all modes: Key 1 (QUALIFICATION) is always mandatory. Without it, Keys 2 and 3 have no reliable reference — the model could declare any domain at query time. This is what PRISM calls the lying model problem.

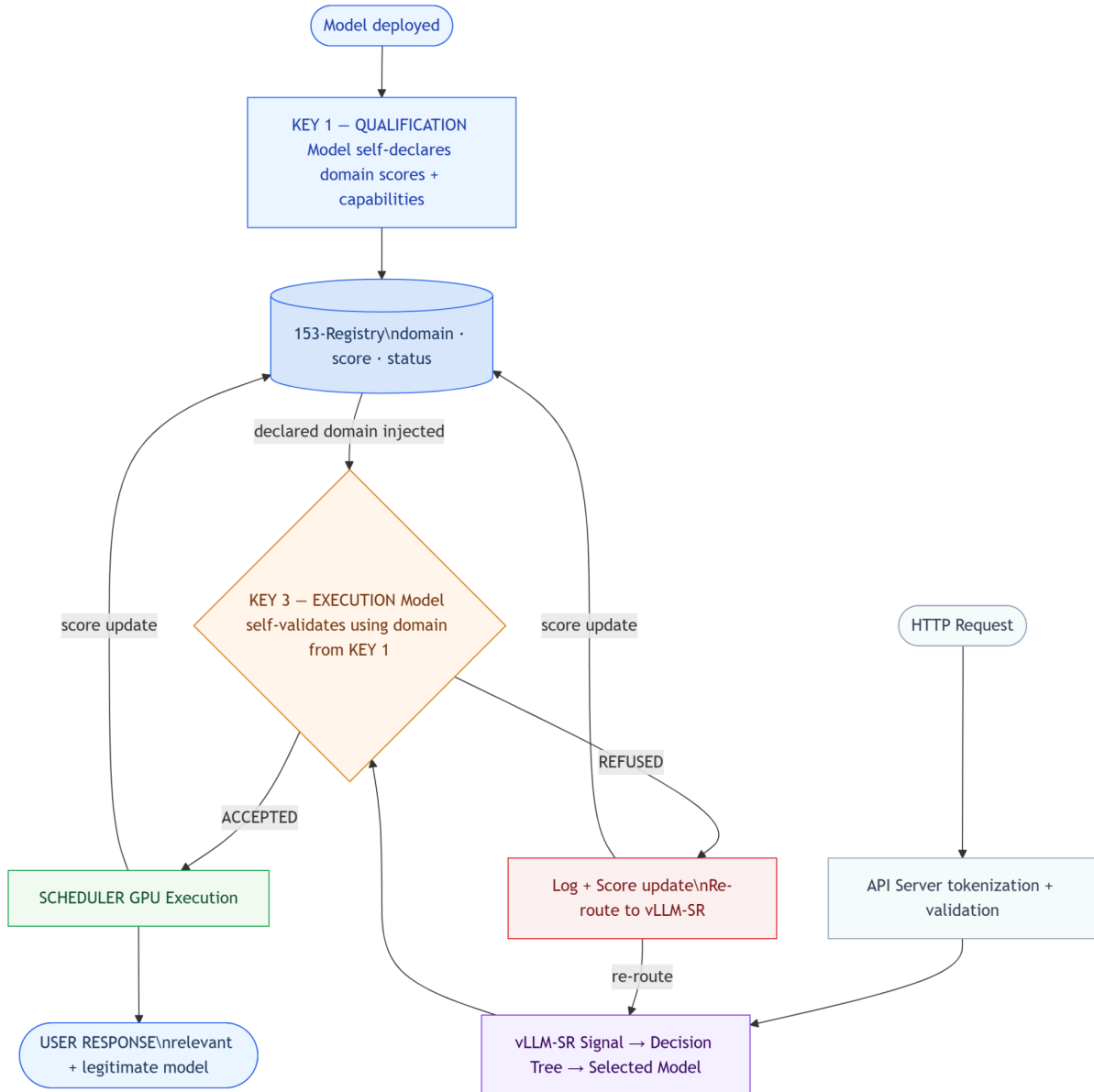
The lying model problem: without a mandatory Key 1 at initialization, a model can declare any domain at Key 3 when the query arrives. The structural constraint comes from the fact that Key 3 injects the domain fixed by Key 1 — not what the model claims to be at runtime.

Mode	Active keys	Principle
fine_filter	Key 1 + Key 3	Qualification at init + forced post-routing validation
coarse_filter	Key 1 + Key 2	Qualification at init + pre-routing intent classification
hybrid	Key 1 + Key 2 + Key 3	All 3 keys — maximum legitimacy guarantee

3.1 fine_filter — Key 1 + Key 3

Active keys: QUALIFICATION (init) + EXECUTION (post-routing)

Key 1 qualifies each model at initialization and fixes its domain in the 153-Registry. At each request, vLLM-SR selects the optimal model, then Key 3 forces that model to confirm its legitimacy using the domain fixed by Key 1 — not what the model could freely declare.

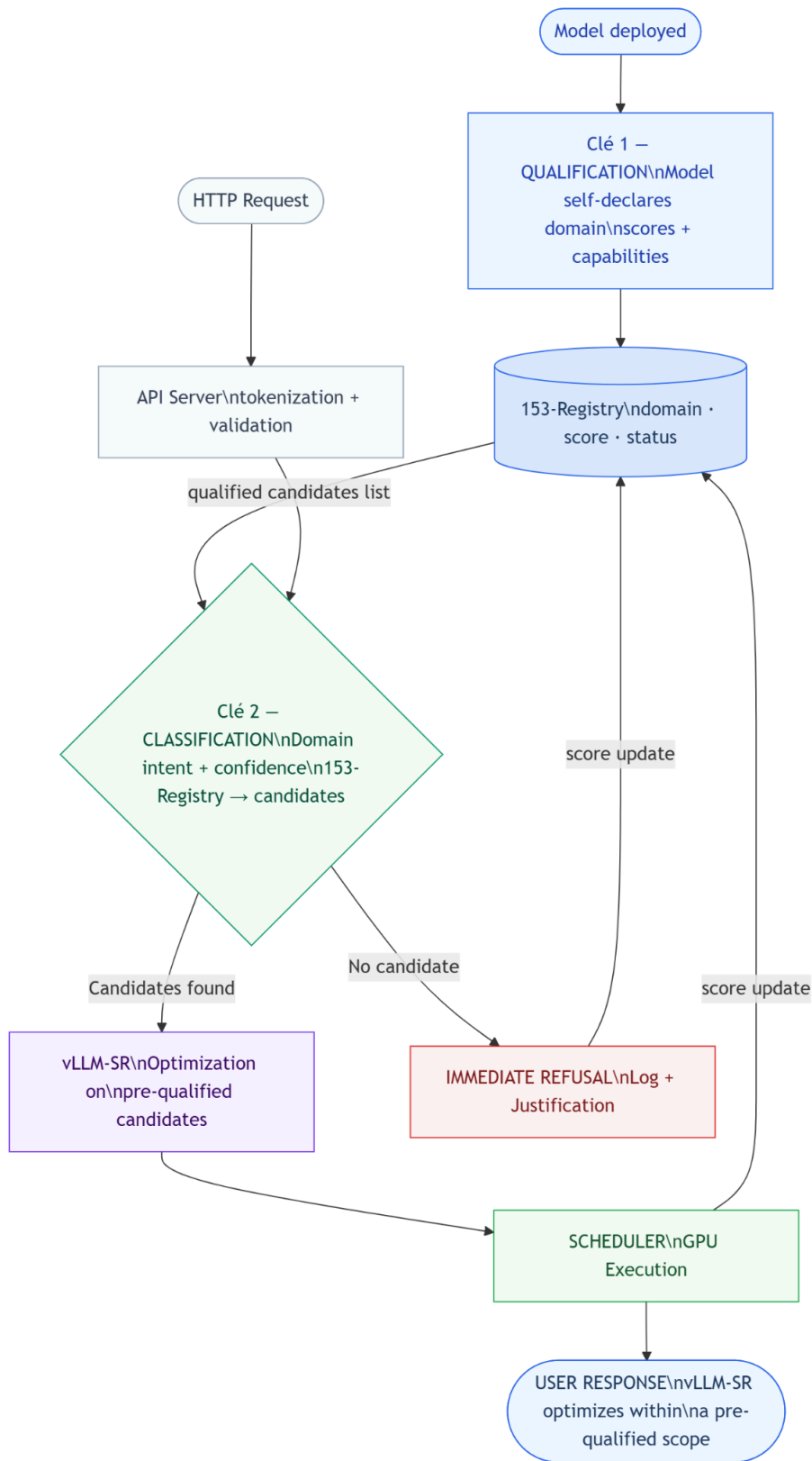


Property	Value
Active keys	Key 1 (init) + Key 3 (runtime)
Integration complexity	Low — no modification to vLLM-SR routing
GPU consumption	Possible before Key 3 refusal
Re-routing on refusal	Yes — model excluded, new candidate selected
Ideal context	Cloud enterprise — Meta, NVIDIA, Google

3.2 coarse_filter — Key 1 + Key 2

Active keys: QUALIFICATION (init) + CLASSIFICATION (pre-routing)

Key 1 qualifies each model at initialization and fixes its domain in the 153-Registry. At each request, Key 2 analyzes the intent and identifies the target domain. The 153-Registry then filters candidates whose domain declared by Key 1 matches — vLLM-SR optimizes only on this pre-qualified list.



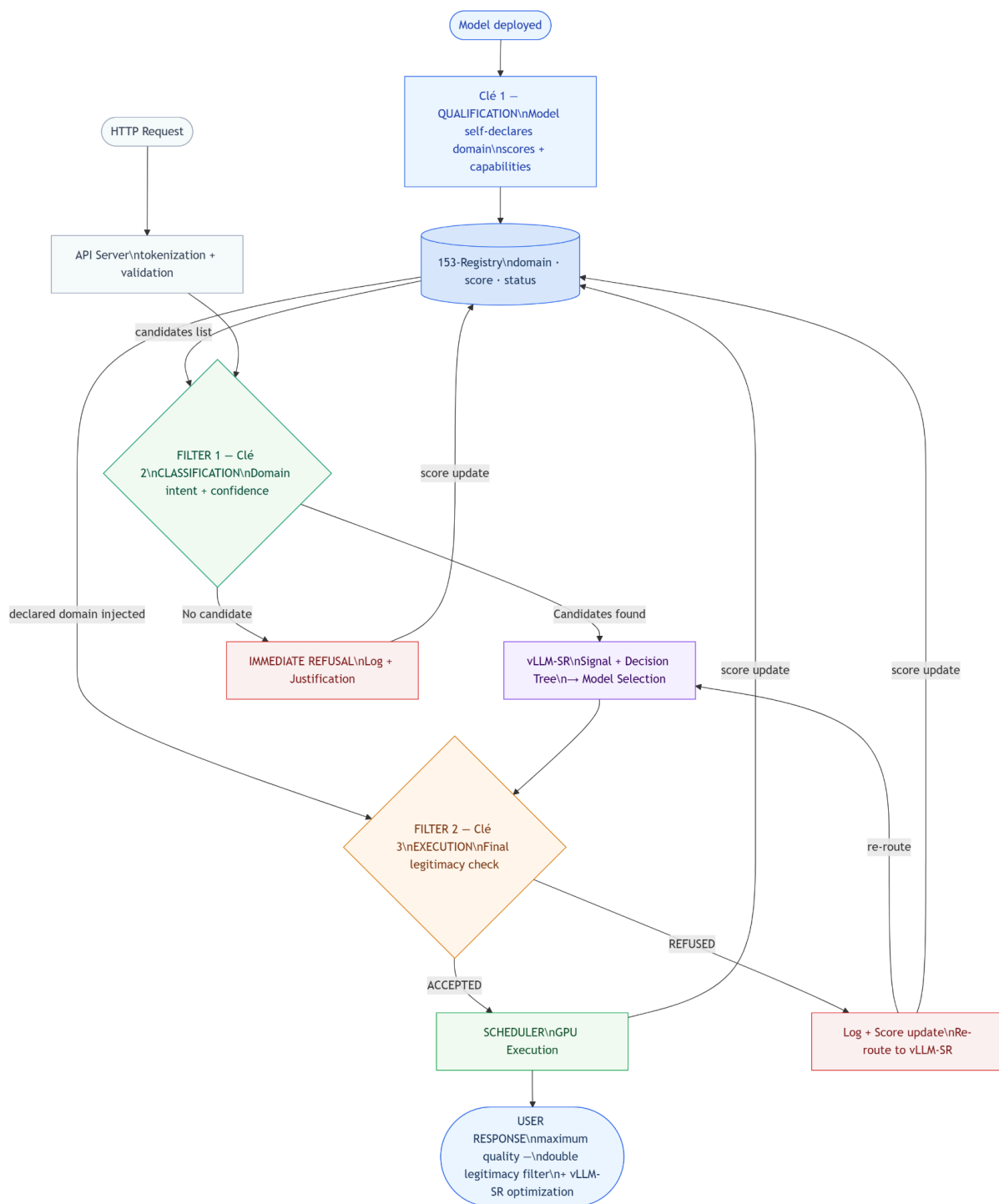
Property	Value
Active keys	Key 1 (init) + Key 2 (runtime)

Property	Value
Integration complexity	Medium — modifies vLLM-SR input pipeline
GPU consumption	Minimized — zero waste on illegitimate requests
Re-routing on refusal	Not applicable — immediate refusal if no candidate
Ideal context	Sovereign on-premise — GDPR, industrial deployments

3.3 hybrid — Key 1 + Key 2 + Key 3

Active keys: QUALIFICATION (init) + CLASSIFICATION (pre-routing) + EXECUTION (post-routing)

Key 1 qualifies each model at initialization. At each request, Key 2 filters legitimate candidates before routing, vLLM-SR selects the best among them, then Key 3 forces a final validation of the selected model. Double guarantee: no unqualified model enters the pipeline, and the selected model structurally confirms its legitimacy before executing.



Property	Value
Active keys	Key 1 (init) + Key 2 + Key 3 (runtime)
Integration complexity	High — three coordinated keys
GPU consumption	Minimized + post-validated by Key 3
Re-routing on refusal	Yes at Key 3 level — model excluded
Ideal context	Critical enterprise — healthcare, legal, finance, GDPR

4. Domain Discovery

A key extension of PRISM beyond static routing is the Domain Discovery mechanism. When Key 1 returns a domain not present in the current 153-Registry, the system does not simply fail — it proposes the creation of a new domain.

4.1 Discovery flow

```
Key 1 QUALIFICATION → domain: "textile_recycling"
|
153-Registry → domain not found
|
Domain Discovery triggered:
  proposed_domain : "textile_recycling"
  model_score     : 0.87
  action          : pending_validation
|
score >= 0.85 → AUTO-APPROVED
score < 0.85 → HUMAN VALIDATION REQUIRED
|
DOMAINS_LIST dynamically enriched
```

4.2 Hybrid validation model

Score range	Action	Rationale
score >= 0.85	Auto-approved	High confidence — genuinely demonstrated expertise
score 0.60 - 0.84	Human validation required	Uncertain — requires expert review before registry update
score < 0.60	Rejected	Insufficient expertise depth

4.3 The completed DNS analogy

Domain Discovery completes the DNS analogy that defines PRISM: *just as DNS allows registration of new domain names, the 153-Registry allows registration of new competency domains.*

The registry is not static — it evolves as new specialized models are deployed. This makes PRISM architecturally self-extending without requiring manual configuration updates.

5. Quality Hypothesis

The combination of vLLM-SR + PRISM may produce higher quality responses than a system without legitimacy routing. The reasoning:

- vLLM-SR selects the model with the highest routing signal relevance
- PRISM validates that the selected model is legitimate to respond within its declared domain
- The model that executes has been qualified on both dimensions simultaneously

This double filter should statistically improve response quality **compared to a system that routes without legitimacy verification**.

This hypothesis remains to be confirmed empirically. The additional latency introduced by PRISM may be justified if it translates into a measurable quality improvement — this is the central trade-off to evaluate in future benchmarking work.

6. Limitations and Open Questions

6.1 Current limitations

- Validation scope: 100+ manual tests on controlled samples — not yet statistically rigorous at scale
- VSLM specification: classification mechanism described but training approach and performance metrics not yet published
- Veracity boundary: PRISM guarantees the legitimacy of the decision to respond — not the correctness of the response itself
- Latency: hybrid mode introduces two PRISM passes — the trade-off with the quality hypothesis requires empirical evaluation

6.2 Open questions for the community

- Which integration mode (fine_filter / coarse_filter / hybrid) is most suitable as a first contribution to vLLM-SR?
- How should Domain Discovery interact with existing capability declarations in ModelConfig?
- Which benchmarks should be used to empirically validate the quality hypothesis?
- Should the 153-Registry be a persistent store or an in-memory structure in the vLLM-SR context?

Reference

Souaissa, M. (2026). PRISM: A DNS-Inspired Semantic Routing Architecture for Distributed Specialized Language Models. Zenodo.

DOI: <https://doi.org/10.5281/zenodo.18750029>

vLLM Semantic Router — Integration issue:

<https://github.com/vllm-project/semantic-router/issues/1422>

Draft PR #1425: <https://github.com/vllm-project/semantic-router/pull/1425>